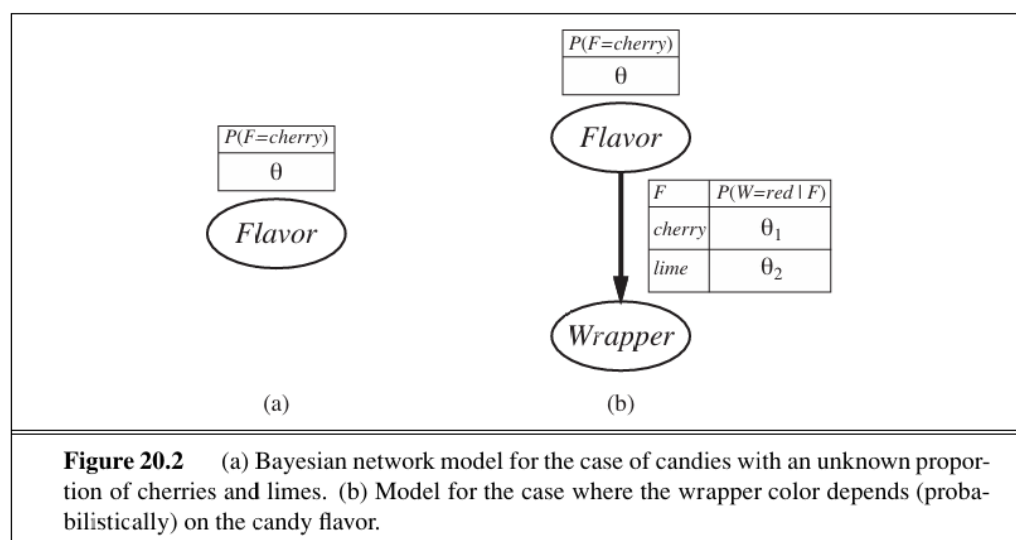


Project #5: Naïve Bayes

Introduction

In machine learning, learning a probabilistic model is a kind of ways to solve tasks. By learning probabilistic theory, we now can construct a probabilistic-based Bayesian network model. The Bayesian network model is a powerful method that provides us with general solutions to solve the problems of noise, overfitting, and optimal prediction. In this project, we have a chance to implement one of the most common Bayesian models called Naïve Bayes. It is 'naïve' because it considers the attributes to be conditionally independent of each other, given the classes. The aim of this project is to create a model that can help you recognize the importance of probabilistic theory in solving classification problems.



(Figure 20.2. Artificial Intelligence. A Modern Approach. P807.)

Objectives

In this project, there're some jobs you need to finish. First, you need to use the code you wrote in project #1 to handle date preprocessing for project #5. Second, you need to implement Naïve Bayes class in *model.py/model.hpp* and train it on the training data. Finally, you need to generate the predict label of the testing data for us to calculate the scoring metric of this project.

(Go check *proj5_guildline_python.pdf* or *proj5_guildline_cpp.pdf* for the further detail)

Dataset Description

The data for this project is real-word data for hospitalizing patients. Each patient is described by 77 *features* (F1~F77), together with a binary outcome. The project involves a training model with provided datasets (train_X.csv & train_y.csv) and final testing on an independent test set (test_X.csv & test_y.csv).

Note:

*In this project, we only provide test_X.csv for you. This means that you cannot directly use the test data to evaluate the model's performance. Therefore, we recommend using **K-fold cross-validation (from scratch)** to decide the best model and **predict the label** for the testing data. We will use your predict label to help you calculate the final prediction score.*

Language:

You can implement your code by using either **Python3** or **C++**. (Just need to submit one of them)

Requirements:

1. Download the necessary library:

This part is the same as project #1.

2. Preprocess the data

Use the code you wrote in the previous project to handle data preprocessing.

3. Implement the detail of Naïve Bayes class:

We have written some start codes framework in *model.py/model.hpp*, please finish your implementation of Naïve Bayes according to it.

Note:

You don't have to follow the start codes in model.py/model.hpp, but we suggest you build your model by following the structure.

4. Predict the label of the testing data

Use your best model to predict the label of the testing data and upload it to E3.

5. Do your implement **from scratch**

Do not use any machine learning libraries to build your model, except for the part we've already done for you in the framework.

Grading Policy (Total 100 pt)

1. Report Spec (75%) (follow this format to write your report)

- A. Explain the concept and code detail of your Naïve Bayes model. (35%)
- B. Questions (30%)
 - (a) Why should we need to fix zero probability problem in Bayesian prediction? If you encounter this problem, how will you solve it? Please provide one example where zero probability might result in wrong answers. (15%)
 - (b) Suppose you are dealing with a dataset where some features are dependent. How will you modify or extend Naïve Bayes algorithm to handle such dependencies? (15%)
(You need to explain your reason, or you may loss some points here)
- C. Discussion (10%)
 - (a) Discuss the relationship between different distributions and Bayesian methods, e.g. Gaussian, Uniform, Poisson Distribution. (10%)
 - (b) Anything else you want to share. (optional)

2. Performance (25%)

For your result of performance, you'll get:

- 25/25: if scoring ≥ 0.63
- 20/25: if $0.63 > \text{scoring} \geq 0.58$
- 15/25: if $0.58 > \text{scoring} \geq 0.53$
- 10/25: if $0.53 > \text{scoring} \geq 0.45$
- 0/25: if scoring < 0.45

$$\text{*Scoring} = 0.3 * \text{acc} + 0.35 * \text{f1 score} + 0.35 * \text{mcc}$$

Performance Metric	Definition
Recall ^a	$TP/(TP + FN)$
Precision ^b	$TP/(TP + FP)$
ACC	$(TP + TN)/(TP + TN + FP + FN)$
F1-score	$\frac{2 \times Recall \times Precision}{Recall + Precision}$
MCC	$\frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP) \times (TP + FN) \times (TN + FP) \times (TN + FN)}}$
AUC	Area under the ROC curve

TP, true positive; TN, true negative; FP, false positive; FN, false negative; MCC, Matthews correlation coefficient; ACC, accuracy; AUC, area under the curve; ROC, receiver operating characteristic.

^a Recall is equivalent to sensitivity in its definition.

^b Precision is equivalent to positive predictive value in its definition.

Note: We will use the predict label you upload to E3 to calculate your performance score. Also, be sure you **set up the random seed** of your best model to avoid controversy.

e.g.

```
import random
random.seed(0)
```

```
import numpy as np
np.random.seed(0)
```

Submission Deadline:

Deadline: 12/27 (Fri.) 23.59. p.m.

Submission format: (Remember to upload your predict label)

- You need to submit **all your code in one folder** together with your **report file, prediction label**, and zip them into one **.zip** file.
- Name your predict label as “pred_hw5_<yourStudentID>_<yourName>.csv”
e.g. “pred_hw5_312551120_陳柏憲.csv”
- Name your code folder as “<language>_hw5_<yourStudentID>_<yourName>”,
e.g. “Cpp_hw5_312551120_陳柏憲” or “Python_hw5_312551120_陳柏憲”
- Name your report file as “report_hw5_<yourStudentID>_<yourName>.pdf”,
e.g. “Report_hw5_312551120_陳柏憲.pdf”
- Name your .zip file as “ml_hw5_<yourStudentID>_<yourName>.zip”,
e.g. “ML_hw5_312551120_陳柏憲.zip”

(Uppercase or lowercase of the letters is fine)

Punishment Policy:

Except for what we mentioned in this file, please refer to [project_policy.pdf](#) we provided in project #1.

We will also run your code in this project, if we **cannot compile and run** your code, we will ask you to re-submit your code, and you will **lose some points** in the final score of this project. (You can tell us how to execute your code and where we should modify the path as the previous project do)

Also, in this project, if your performance metrics are very similar to your classmate's, we may suspect cheating and check your code very carefully. Maybe we will ask you and your classmate to explain it in person. Please be aware!

Good luck~

TAs